

Object-Oriented Relationships in C++

Association • Aggregation • Composition • UML Diagrams

Understanding how objects interact in real-world systems.
(Pre-Final Project Concept Refresher)



Quick Recap: OOP Concepts

- **Classes / Objects** → encapsulate data & behavior
- **Inheritance** → “is-a” hierarchy (reuse & extension)
- **Polymorphism** → flexible interfaces
- **Virtual functions** → enable runtime behavior changes
- **Why Class Relationships Matter**
 - Real-world systems = multiple objects interacting
 - Helps identify ownership and lifetime dependencies
 - Crucial for **software architecture & UML modeling**
 - Makes your final project designs more modular and maintainable



Association Relationship in Classes

- A *general relationship* between classes — both can exist independently.
 - A Patient is *associated* with a Doctor.
 - They can exist independently — deleting one doesn't destroy the other.
 - Relationship often implemented via **pointers** or **references**.
- **UML Notation:**
 - Solid line with arrow (optional direction)
 - *A Patient is treated by a Doctor.*

```
class Doctor {  
public:  
    std::string name;  
    void treat(Patient* p);  
};  
  
class Patient {  
public:  
    void assignDoctor(Doctor* d) { doctor = d; }  
private:  
    Doctor* doctor; // Association (link, not ownership)  
};
```

Aggregation Relationship in Classes

- A “*has-a*” relationship where the container holds references to independent objects.
 - Department aggregates Course objects.
 - Courses can exist even if the Department is deleted (no ownership).
 - Common in manager–component or controller–model setups.
- **UML Notation:**
 - White diamond at the “whole” end
 - *Department* o— *Course*

```
class Course {  
public:  
    std::string name;  
};  
  
class Department {  
public:  
    void addCourse(Course* c) { courses.push_back(c); }  
private:  
    std::vector<Course*> courses; // Aggregation  
};
```

Composition Relationship in Classes

- A *strong ownership* relationship — parts can't exist without the whole.
 - Car *owns* its Engine.
 - When the Car is destroyed, the Engine is too.
 - Represents *part-whole* dependency.
- **UML Notation:**
 - Black diamond at the “whole” end
 - Car  Engine

```
class Engine {
public:
    Engine(int hp): horsepower(hp) {}
    void start() { std::cout << "Engine running\n"; }
private:
    int horsepower;
};

class Car {
public:
    Car(std::string brand): brand(brand), engine(250) {}
    void drive() { engine.start(); }
private:
    std::string brand;
    Engine engine; // Composition
};
```

UML Relationship Summary

Relationship	Lifetime Dependency	UML Symbol	Example
Association	None	Simple line	Doctor–Patient
Aggregation	Independent	○ (white diamond)	Department–Course
Composition	Dependent	◆ (black diamond)	Car–Engine

UML Class Diagram Overview

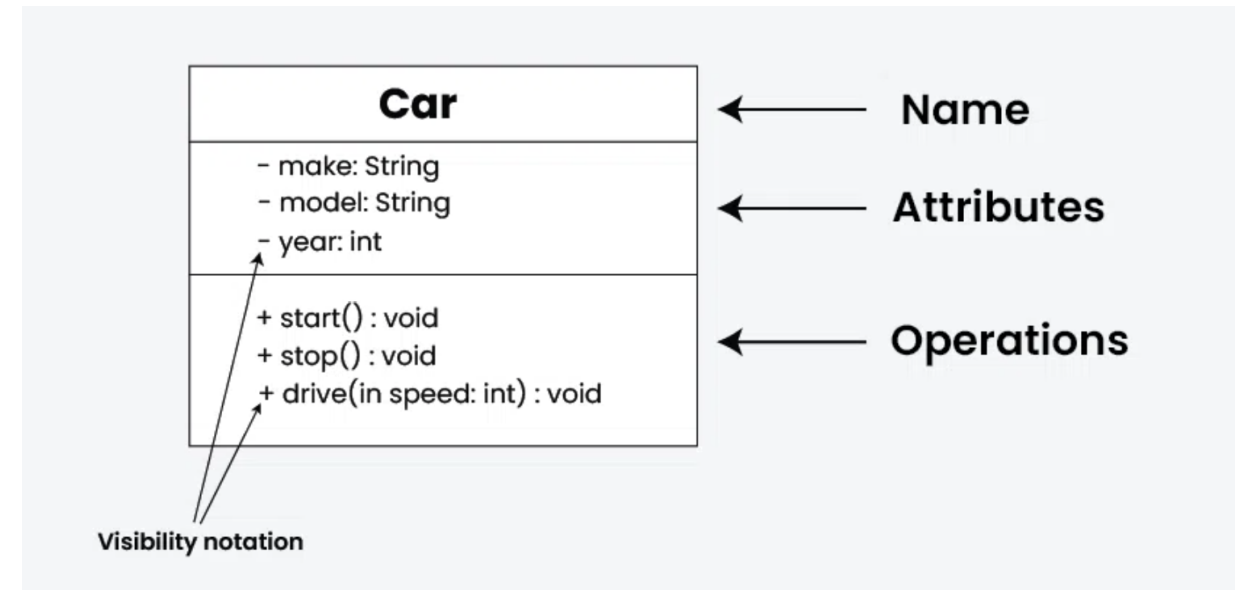


What is a UML Class Diagram?

- UML class diagram visually represents the **structure** of a system by showing:
- **Classes**, their **attributes** and **methods**
- **Relationships** between classes
- **Why it matters:**
 - Helps everyone (developers, designers, PMs) understand how the system is organized
 - Clarifies how components **interact**
 - Acts as documentation of system structure
 - *Think of it as a “blueprint” for your software system.*

UML Class Diagram

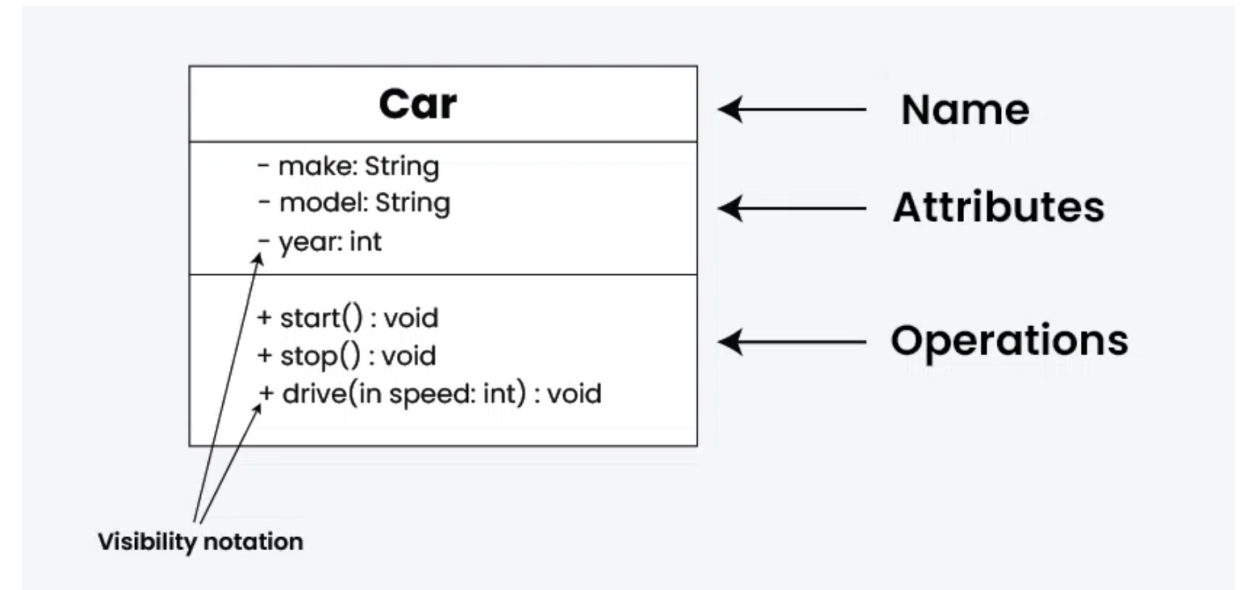
- **UML Class Notation**
- **Classes are drawn as boxes with three compartments:**
- **Top:** *Class Name* (centered, bold)
- **Middle:** *Attributes* (data members)
- **Bottom:** *Methods* (functions or operations)



UML Class Diagram

• Visibility notations

Symbol	Meaning
+	Public
-	Private
#	Protected
~	Package (default)



UML Class Diagram

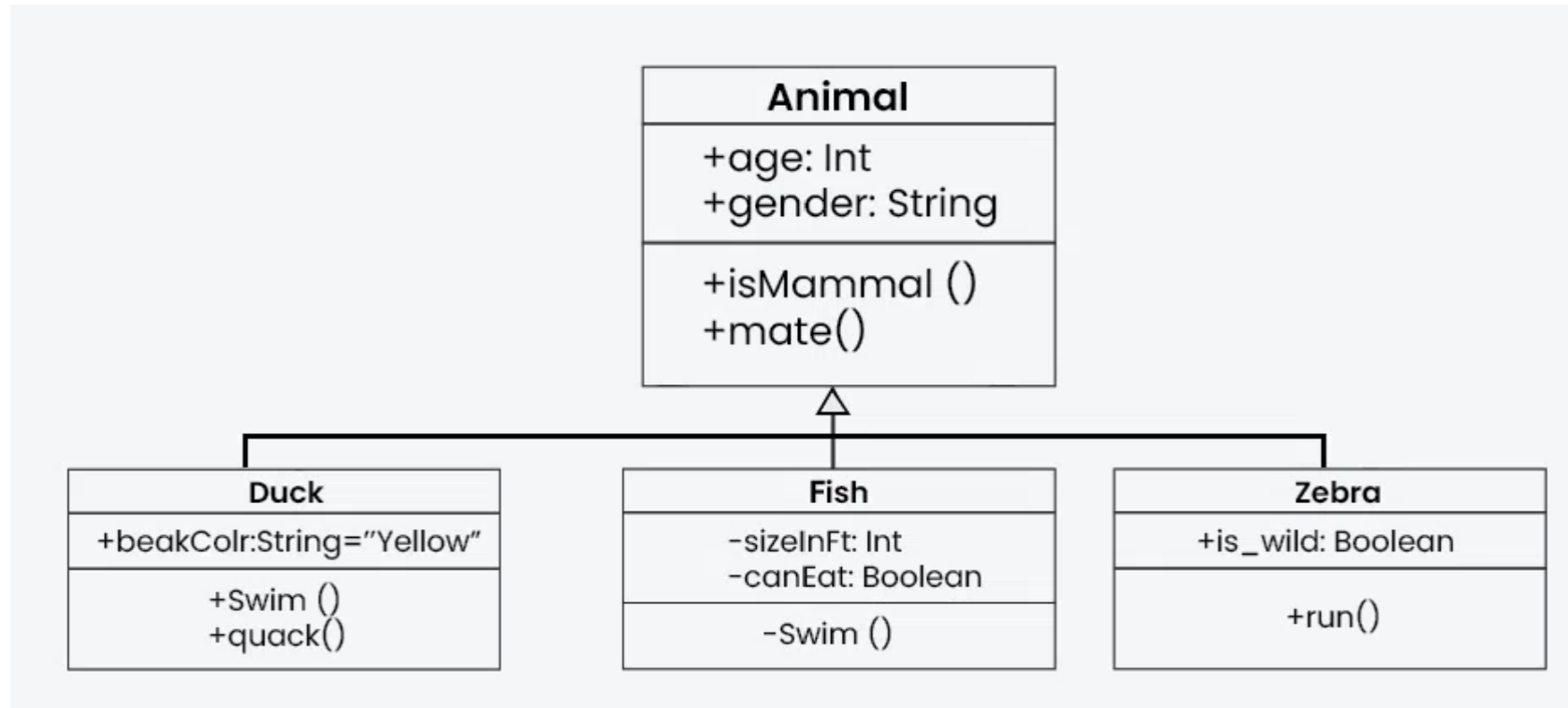
- Parameter direction shows the **flow of data** between classes during method calls.
- This helps clarify how data moves across classes.
- Example: `+printDocument(in fileName: String, out status: Boolean)`

Direction	Description	UML Symbol
<code>in</code>	Input to a method	→ toward the class
<code>out</code>	Output from a method	← away from the class
<code>inout</code>	Both input and output	↔ both directions

Relationships Between Classes

Type	Description	UML Symbol
Association	Basic connection between two classes	Solid line
Directed Association	Connection with a defined direction	Solid line with arrow
Aggregation	"Whole-Part" (parts exist independently)	○ White diamond
Composition	"Whole-Part" (parts depend on the whole)	◆ Black diamond
Generalization	Inheritance ("is-a")	Hollow arrow
Dependency / Usage	Class uses another class temporarily	Dashed arrow line

UML Diagram Example





Benefits of UML Diagrams

- **Purpose:**

- Clearly depicts OOP concepts: inheritance, abstraction, relationships
- Provides foundation for deployment and component diagrams
- Supports **forward and reverse engineering**

- **Benefits:**

- Visualizes software architecture
- Clarifies class responsibilities and connections
- Improves communication between technical and non-technical team members
- Guides coding and ensures design–implementation alignment
- Enables automated **code generation** in many IDEs



SFML (Simple and Fast Multimedia Library)

- A modern C++ library for **graphics, games, and interactive apps**. Think of it as a simpler way to do what OpenGL or SDL can — but with a clean, object-oriented design.
- **Core Modules**
 - `sf::System` – Time, threads, basic utilities
 - `sf::Window` – Event handling, window creation
 - `sf::Graphics` – Shapes, text, textures, sprites
 - `sf::Audio` – Sound and music
 - `sf::Network` – Networking (optional)
- **Why It Fits This Course**
 - Uses **classes, objects, inheritance** → reinforces OOP.
 - Lightweight, cross-platform, and easy to install.
 - Great for **hands-on visual learning** — see your C++ code in action!
 - Opens the door to **game loops, events, and animations**.



How to Navigate SFML

- [Tutorials \(SFML / Learn\)](http://www.sfml-dev.org/tutorials/2.6/) (<http://www.sfml-dev.org/tutorials/2.6/>)
- SFML is well-structured and object-oriented — every concept is a **class**, and each class is part of one of five **modules**:
system, window, graphics, audio, network.
- **Know the Class You Need**
 - Want to **draw something**? → Go to **Graphics Module**
 - Want to **detect input or close windows**? → Go to **Window Module**
 - Want to **play sound**? → Go to **Audio Module**
- Example let's draw a circle shape:
 - `sf::CircleShape`
 - inherits from `sf::Shape` → which inherits from `sf::Drawable`
 - `sf::Drawable` can be drawn with `window.draw(...)`
- Reinforces **class hierarchy navigation** + OOP understanding.



How to Navigate SFML

- **se the Official Tutorials (Before the API Docs)**
- The SFML tutorials are **the fastest way** to get productive:
<https://www.sfml-dev.org/tutorials/2.6/>
- Recommend starting with:
 - **Graphics → Drawing shapes**
 - **Window → Events and input**
 - **System → Time and clock (for animations)**
- Tip: Don't memorize, browse intentionally. Each tutorial has **short examples** that compile directly